Lecture 4

Data and Transaction Design

Data and Transaction Design

Outlines

- Logical Data Architecture
- Logical Data Viewpoint
- Transaction Design

Logical Data Architecture

- The logical data architecture (data schema/logical data model) describes the structure of entities, relationships, and constraints used in the software.
- A data schema has two main applications/ purposes :
  1. **Persistent storage model:** the relationship between data schema and their mapping to persistent storage
     - Defines how data is stored in databases (SQL, NoSQL, files, etc.).
  2. **Messaging model:** define individual messages and their relationships
     - Specifies the format of data exchanged between components (APIs, events, etc.).

Logical Data Architecture…

Modeling Approaches

- The data schema may be modeled as using:
  - **Object model** or
  - **Entity-relationship (ER) models**.
- **Object models:** Used for modeling non-relational data bases such as NoSQL, files and other object-oriented systems.
  - ❖ well-suited for representing complex data structures and relationships,
- **Entity-Relationship (ER) Models**: Common in relational database design.
  - ❖ focusing on entities (objects), their attributes (properties), and the relationships between them

Logical Data Architecture…

Logical data model stability

Challenges in Modifying Data Models

Once a system is deployed, changes to the data model can be impacts on:

- **Budget & Schedule** – Requires rework in storage, APIs, and UI.
- **Testing** – Extensive validation needed for backward compatibility.
- **Customer Interface** – Changes may break existing integrations.

- To see why modifications are difficult, consider the possible types of data model changes:
  - Add or remove a class/ attribute
  - Change an attribute type
  - Move attribute to different class
  - Restructure a class hierarchy

Logical Data Architecture…

Logical data model stability

- For example, If customers email is added as a new attribute, then customer user interface need to modified.

  Example: Adding an email attribute to a Customer class requires:

  - Database schema updates
  - API contract changes
  - UI modifications

- Migrating to a new hardware, new software requires extensive testing to ensure the resulting data is correct to that specific version.

Logical Data Architecture…

Effects of the stable logical data model

Benefits of a Stable Data Model

- Facilitates rapid and early custom report development
- Enables a good integration of components
- Allows adding new functions without impacting existing systems

To achieve a stable data model:

- **Architecture Team Involvement:** For the development of the data architecture
- **Frequent meeting:** Regular meetings and communication among stakeholders
- **Modern Data Architecture Decisions:** Decisions about using appropriate data architectures
- **Architecture Approvals:** Formal approval processes for the data architecture

Logical Data Viewpoint

- The Logical Data Viewpoint provides an overview of all the major data entities in the system.
- A logical data view is a way of looking at the data model of a system from a specific perspective.
- Instead of looking at *all* the data and its relationships, a data view focuses on the data relevant to a specific part of the system or a specific function

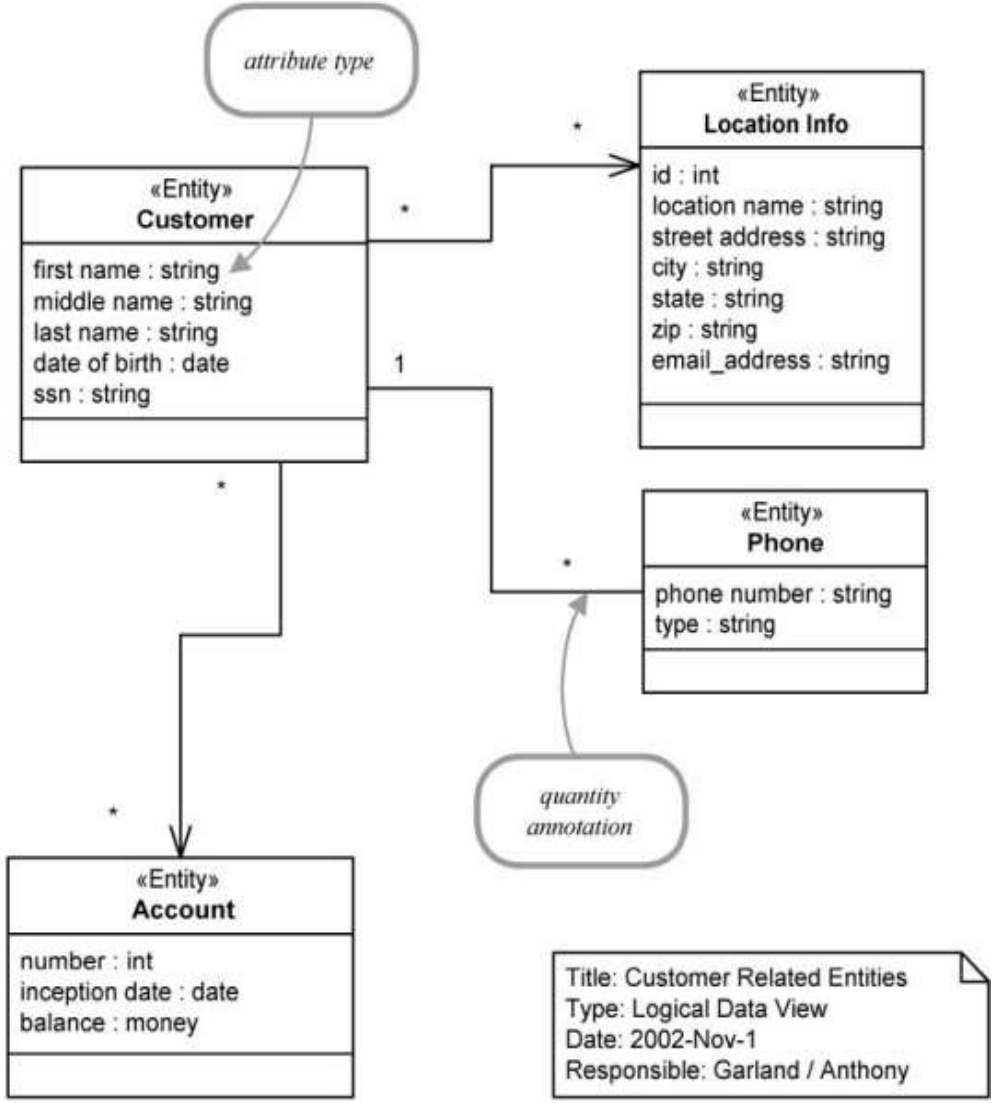
Logical Data Viewpoint

Example 1: As shown in the figure:

- The logical data model initially designed for a specific country.
- This design might include fields like "State" and "Zip Code," which are common in U.S. addresses.

The problem:

- When the system needs to support international addresses, these fields might not be relevant or accurate for addresses in other countries (E.g Addresses in Europe do not have a state or zip code).
- Many customers may have several email addresses (e.g., work, personal) at the same location., and an email address for a portable device like a mobile phone that doesn't really relate to a location (accessible anywhere the device goes).



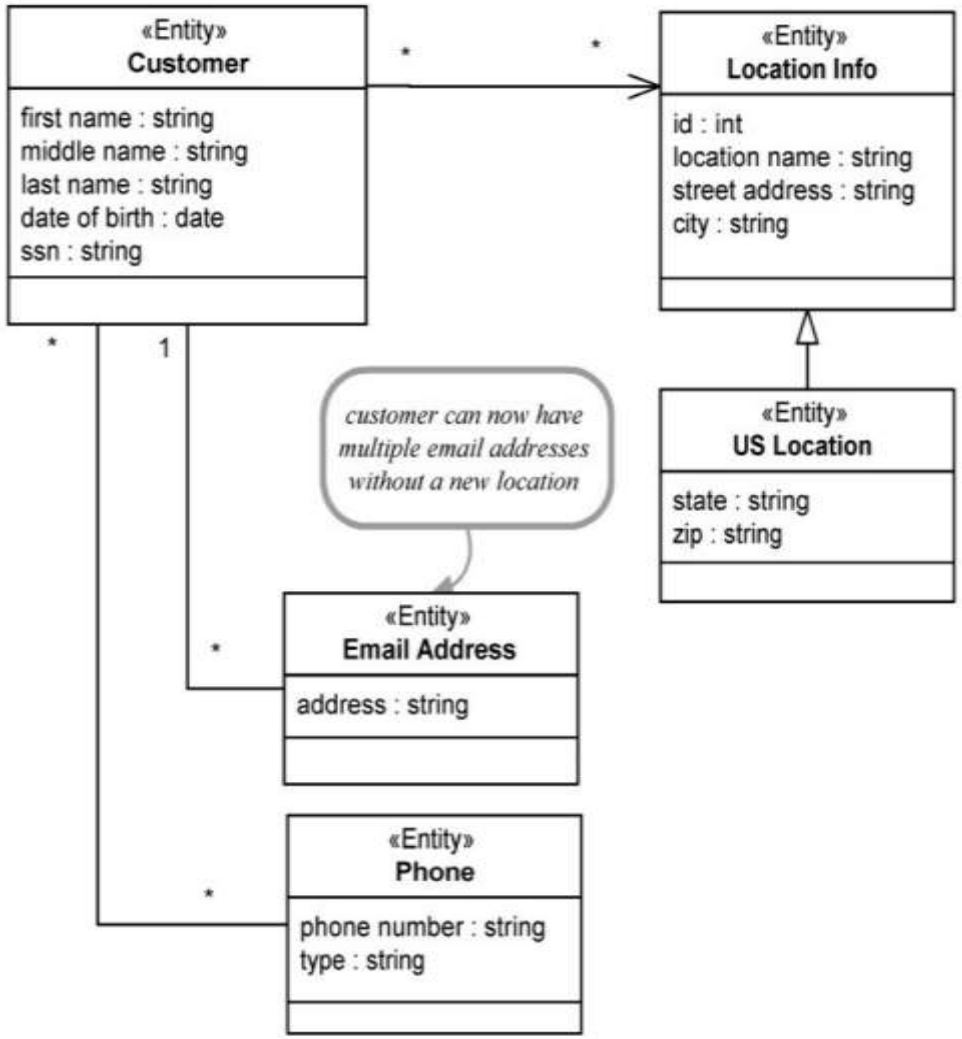
Example subset of Logical Data View

Logical Data Viewpoint

The solution: refactoring

- The logical data view needs to be "refactored" (restructured or redesigned) to accommodate these differences.
- This might involve: Making "State" and "Zip Code" optional.
- Allowing a customer to have a collection of email addresses instead of a single field.
- Creating a separate entity or relationship to link email addresses to specific Phone devices.

- This refactoring clarifies the relationships between customers, their email addresses, and the devices they use.



Refactored Logical Data View

Logical Data Viewpoint

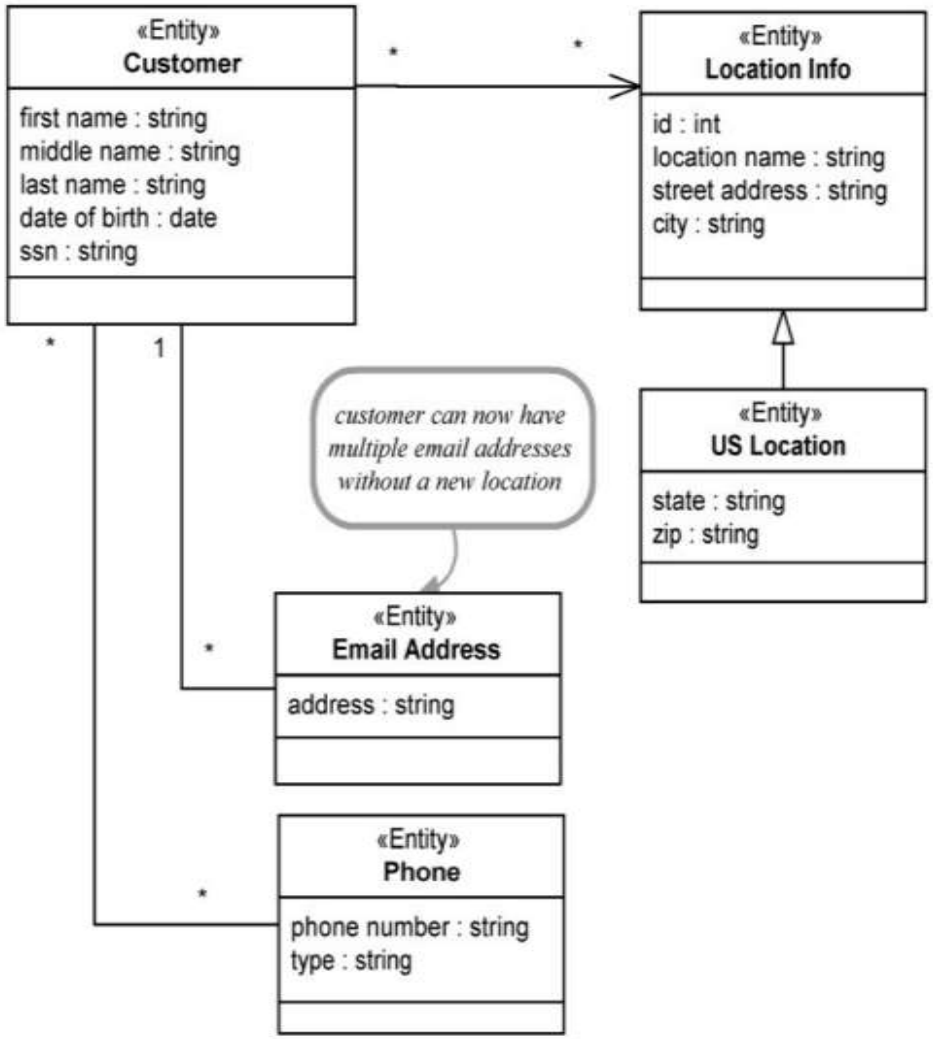
- If the data in the repository or message set is only used within a single subsystem and is not shared with other parts of the system, then this data becomes part of a subsystem design.

Why this is important:

- It promotes modularity, Each subsystem becomes more independent.
- It simplifies the design, Subsystems don't need to worry about the complexities of data used by other subsystems.
- It encapsulates data, Changes within a subsystem's data view are less likely to affect other parts of the system.

Logical Data View for messaging

- When data is exchanged through messages, the structure of those messages is often simpler compared to how data is stored persistently.
  - Messages tend to have fewer complex relationships (associations and aggregations).
- Messages often utilize rich inheritance structures. This allows for flexibility in message handling..
- This allows messages to be sent, received, and routed to message handlers by general-purpose frameworks.



Refactored Logical Data View

Data Model Design – Other Considerations

- Several other important considerations come into play when designing data models.
- These include :
  - Interaction between data models and architectural layers,
  - Use of reflection for dynamic data models and
  - Mapping of objects to relational databases impacts architectural decisions.

Data Model Design – Other Considerations…

Data models and layers

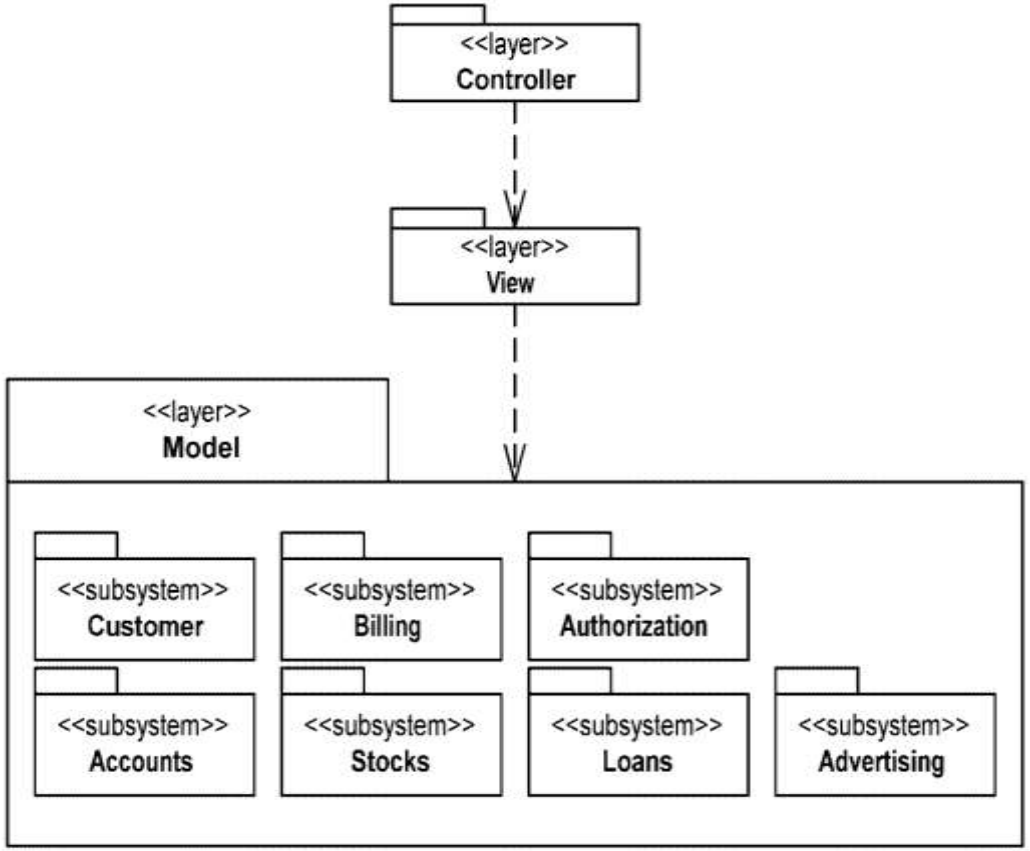
- Data models are often closely tied to the architectural layering of a system.
- Architectural layering is a design pattern that organizes the system into distinct layers, each with specific responsibilities.
- This organization helps to improve modularity, maintainability, and separation of concerns.

  E.g, In the **Model–View–Controller Pattern** the data model is usually a large part (a core part) of the model layer.
- The model layer is responsible for managing the application's data and business logic.

Data Model Design – Other Considerations…

Data models and layers

- Entities within the model layer are frequently stored in a persistent store or transmitted in messages.
- In a layered architecture, the subsystems associated with the data model/core of the layered architecture.
- Modifications to the model subsystems can be costly, potentially requiring changes to upper layers of the architecture to maintain consistency with the data changes.
- In contrast, modifications to the view or control layers typically have a less significant impact on the overall architecture.



Example layering strategy

Data Model Design – Other Considerations…

Data models and reflection

- Reflection is a powerful technique that allows systems to achieve flexible data models.
- It provides software with the ability to examine and modify the schema of data at runtime and adapt its behavior accordingly
    - E.g. ORM (Object-Relational Mapping) frameworks like Hibernate use reflection to map database tables to Java objects at runtime
- Reflection facilitates the implementation of infrastructure software that provides essential services such as marshaling (converting data between different formats), parsing (analyzing data structures), and serialization (storing/transmitting objects).
- Reflection is an important architectural design strategy that can used to improve the adaptability of systems.

Data Model Design – Other Considerations…

Mapping objects to relational database

- Mapping objects to a relational database is a common architectural issue today because of the extensive use of object-oriented design and relational database systems.

Case 1: Difference in Identifiers

- Mapping objects to a relational database is challenging due to the difference in identifiers for objects (object ID) and database (attribute key values as ID).
- Object-relational mapping (ORM) frameworks and technologies,
  - such as Java Persistence API, Entity Beans, and others, are designed to bridge this gap by providing mechanisms to map object identifiers to database keys.

Data Model Design – Other Considerations…

Mapping objects to relational database

Case 2: The relationship between the database tables and the objects

- Another significant challenge lies in mapping the relationships between database tables and the objects in the object model.
- For example, each primitive attribute (data field) in an object model(class) must be represented by a corresponding column in a database table.
- There are at least three major approaches for mapping **tables to classes**:
  - **One table for each class:** Each class in the object model is mapped to a separate table in the DB.
  - **One table for each concrete class:** Only concrete (non-abstract) classes are mapped to tables.
  - **One table for all classes:** All classes in an inheritance hierarchy are mapped to a single table.
  - **Several tables representing a generic data model:** A more complex mapping scheme using multiple tables to represent the data.
- Technologies like LiNQ (Language Integrated Query) & DAL (Data access layer for Code organization) in C# provide tools and mechanisms to facilitate these different mapping approaches.

Data Model Design – Other Considerations…

Mapping objects to relational database

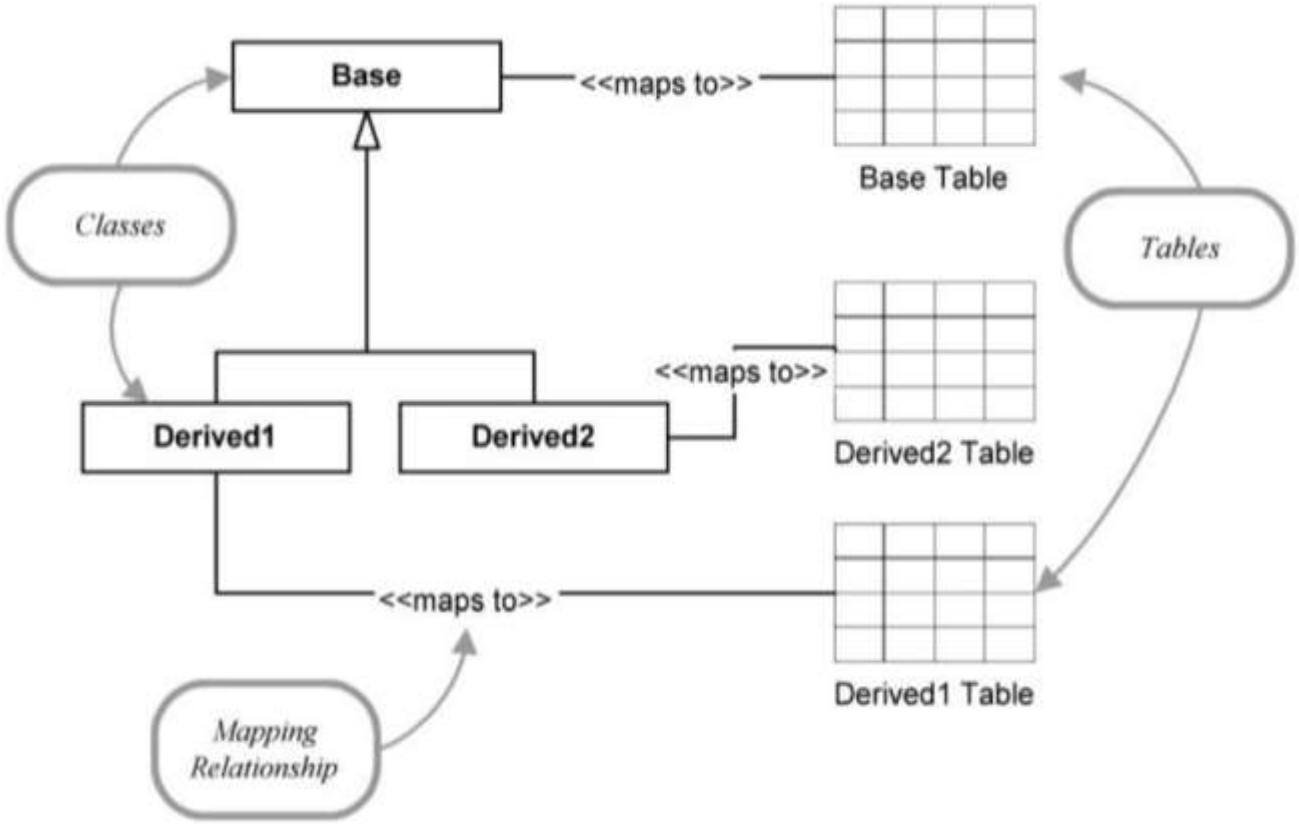


Fig. Class to table mapping design

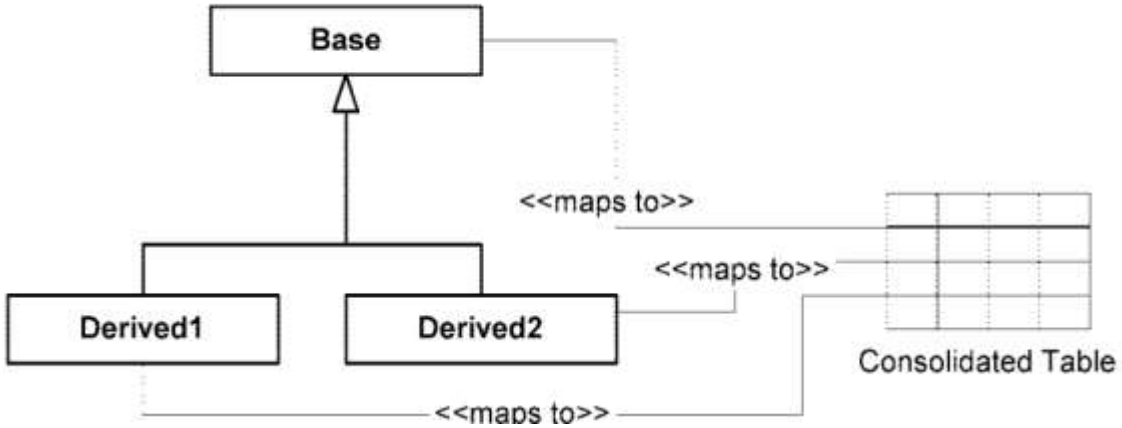


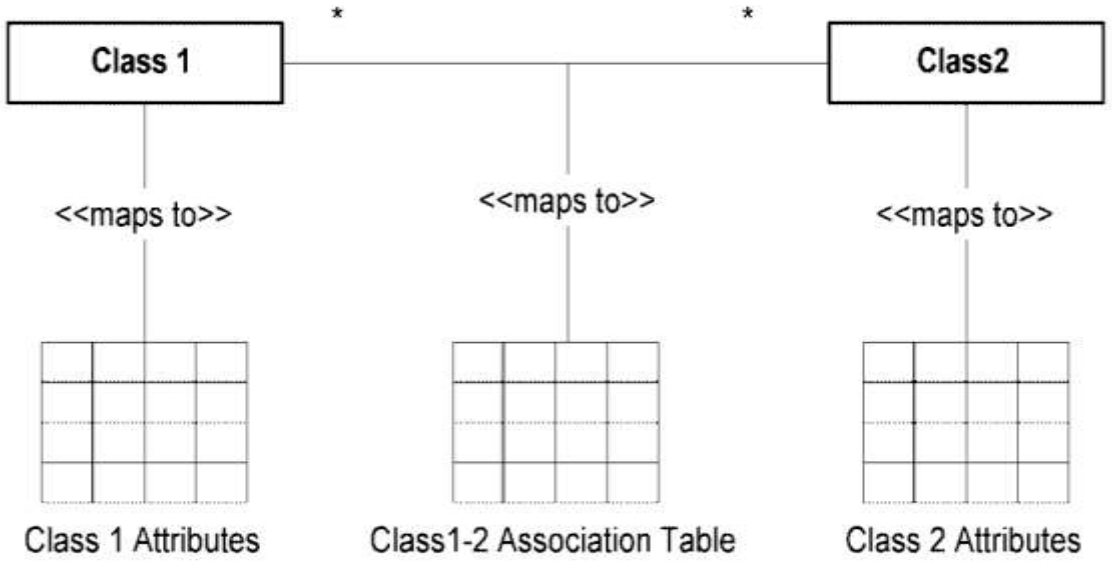
Fig. Alternative class to table mapping design

Data Model Design – Other Considerations…

Mapping objects to relational database

Case 3: Managing relationships between objects

- Another challenge for object to relational database mapping is managing relationships between objects.
- For example,
  - **Class 1** maps all of its attributes to a single table,
  - **Class 2** maps all of its attributes to a table,
  - The association between the classes maps to a third relational table.



Transaction Design

- Another critical architectural design issue is the understanding of transactions and component interaction concurrency.
- There are several reasons for this fact.
  - First, poorly designed transactions can result in incomplete database state
  - Second, transactions typically span across subsystem, layer, component, and even process boundaries.
    - Thus, the correct management of the transactional state of data is essential for ensuring the correct behavior of many operations within a system.
- Key architectural issues to consider in transaction design include: :
  - Which component(s) should be responsible for controlling transactions?
  - How do transactions relate to the interfaces of components?
  - Where are potential concurrency hotspots/*issues* in the system?
  - What are the expected transaction rates for updates to the data?

Transaction Design…

Transaction concepts

- **Transactions** are a unit of interaction with a persistence mechanism.
- Most database systems provide **ACID properties**::
  - **Atomicity** – A transaction is an indivisible unit of work; it either executes completely, or it has no effect at all. It is never partially executed.
  - **Consistency** – transaction maintains the consistency of the database, ensuring that it transitions from one valid state to another valid state. Data is never half-committed.
  - **Isolation** – Transactions are isolated from each other, meaning that concurrent transactions do not interfere with each other's execution. Transactions are independent of each other
  - **Durability** – Once a transaction is committed, its changes are permanent. Data can survive process failure such as power outages or crashes

Transaction Design…

Transaction concepts

- **Locking and locking granularity** are important considerations in database architecture.
- Locking mechanisms prevent concurrent access to data to avoid conflicts.
  - e.g., locking an entire database, a table, a row, or even a specific data value.
- The choice of locking granularity significantly impacts performance.
- A frequent design goal when using databases is to keep update transactions as short as possible and allowing multiple components to write to the database without blocking for significant periods of time.
- **Transaction and Concurrency** need coordination to avoid inconsistency

Transaction Design…

Transaction concepts

*The **level at which locks are applied** significantly impacts **performance** and **concurrency**.*

Granularity Level	Description	Pros & Cons
Database-Level Lock	Locks the entire database.	✓ Simple to implement. ✗ Severely limits concurrency.
Table-Level Lock	Locks an entire table.	✓ Good for bulk operations. ✗ Blocks other transactions on the same table.
Row-Level Lock	Locks a single row.	✓ High concurrency. ✗ Higher overhead than table locks.
Field-Level Lock	Locks a specific column value.	✓ Maximum granularity. ✗ Rarely used due to complexity.

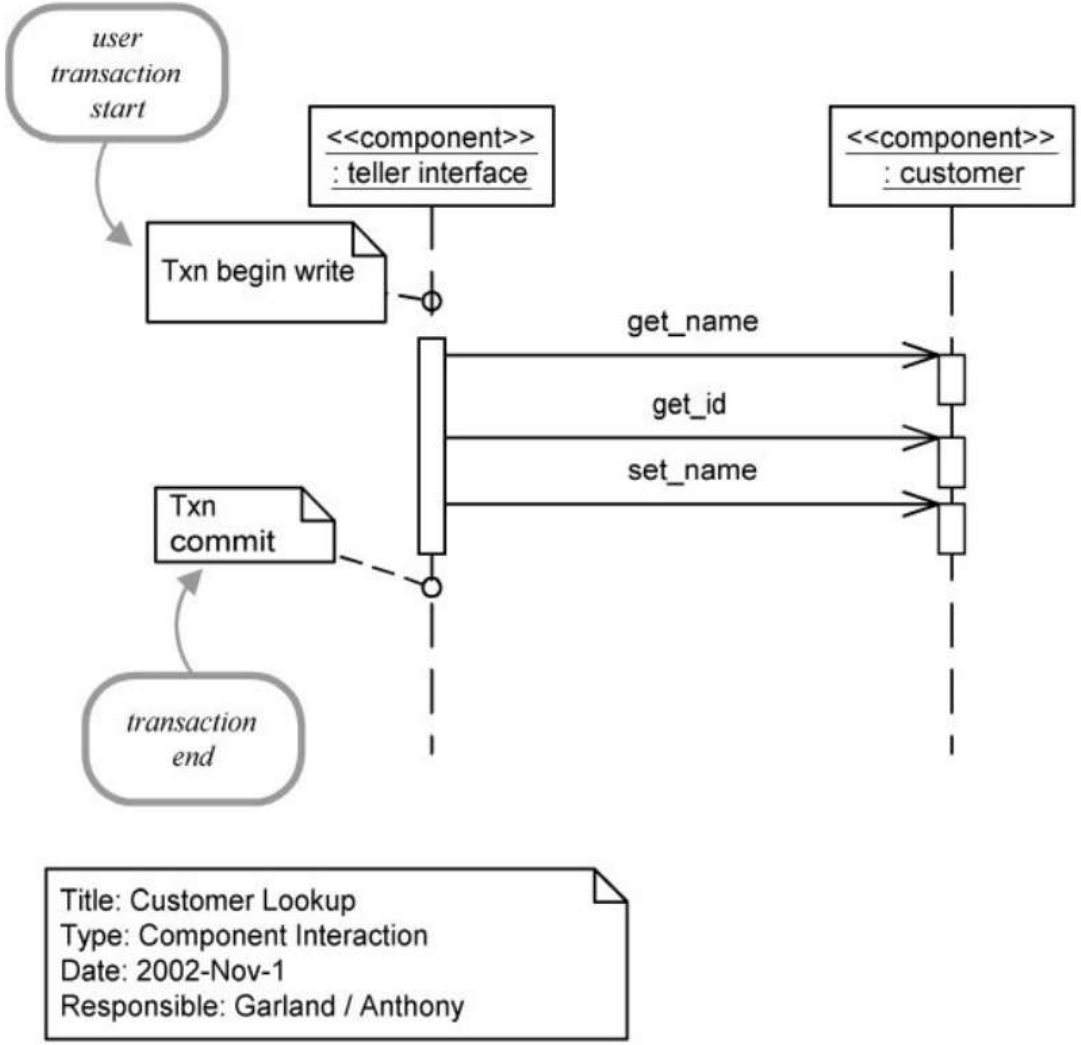
Example Scenarios

- ***Banking System**:*
  - ***Row-level lock** on an Account balance during a transfer.*
  - *Prevents two transactions from modifying the same account simultaneously.*

Transaction Design…

Modeling transaction dynamics

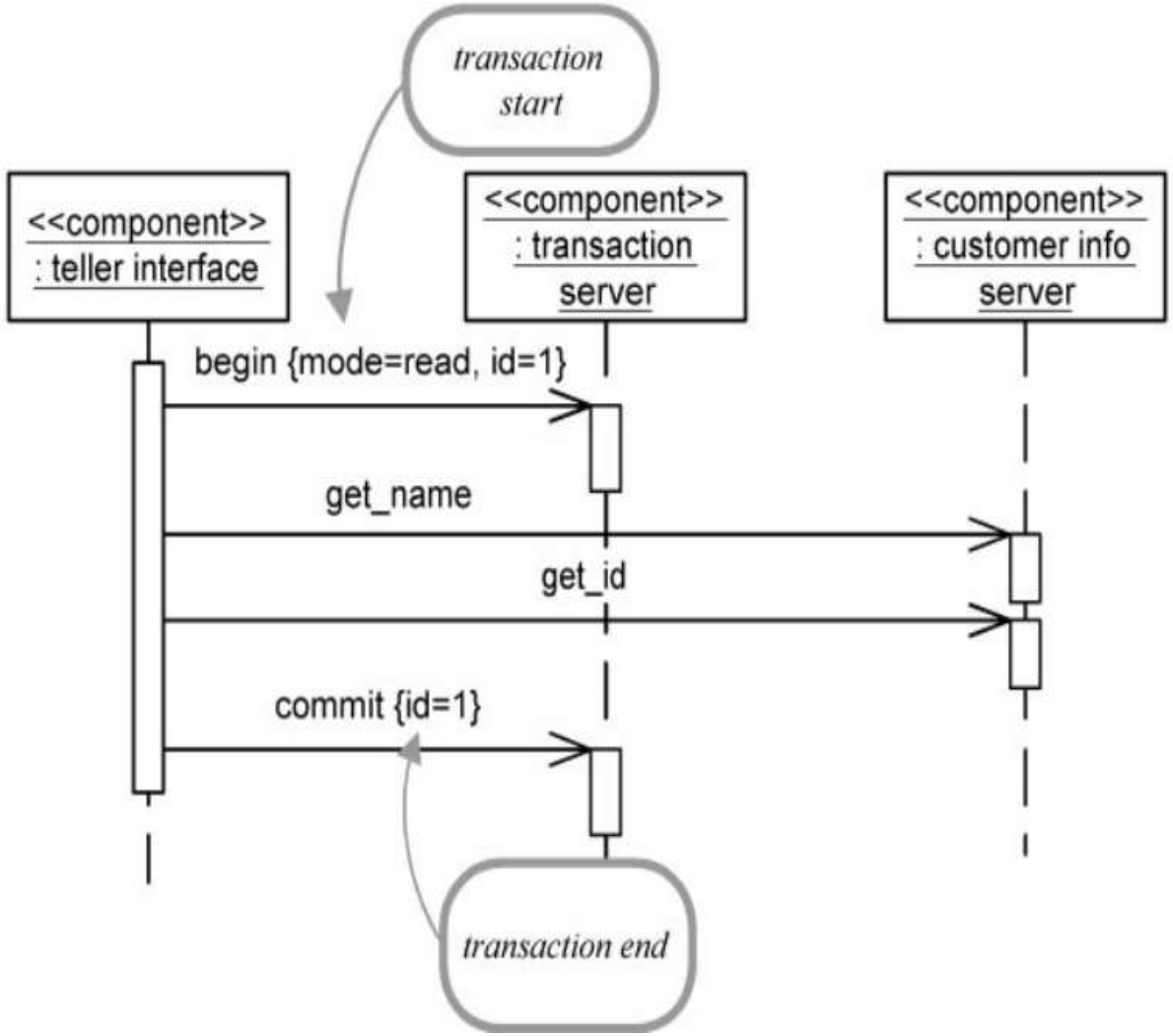
- The first step in transaction design is to understand where transactions fit into the system's dynamic model.
- There are at least two approaches to model transaction dynamics:
  - Analysis Interaction View (user perspective on the transaction):focusing on the high-level steps involved.
  - Component Interaction View (component perspective on the transaction):showing how different components interact and which operations they perform.



Transaction Design…

Modeling transaction dynamics

- Tagged values can be used in models to indicate the transaction mode (e.g., read-only, read-write) and to assign a transaction identifier.
- The transaction identifier is particularly useful when modeling nested transactions or transaction abort scenarios.
- Dynamic models help to clearly identify which components are responsible for initiating, managing & completing transactions.



Thanks!